



경북대학교 IDEC 2026 창 의 회로설계 챌린지 디지털시스템 설계 분야 교육

June. 30, 2026

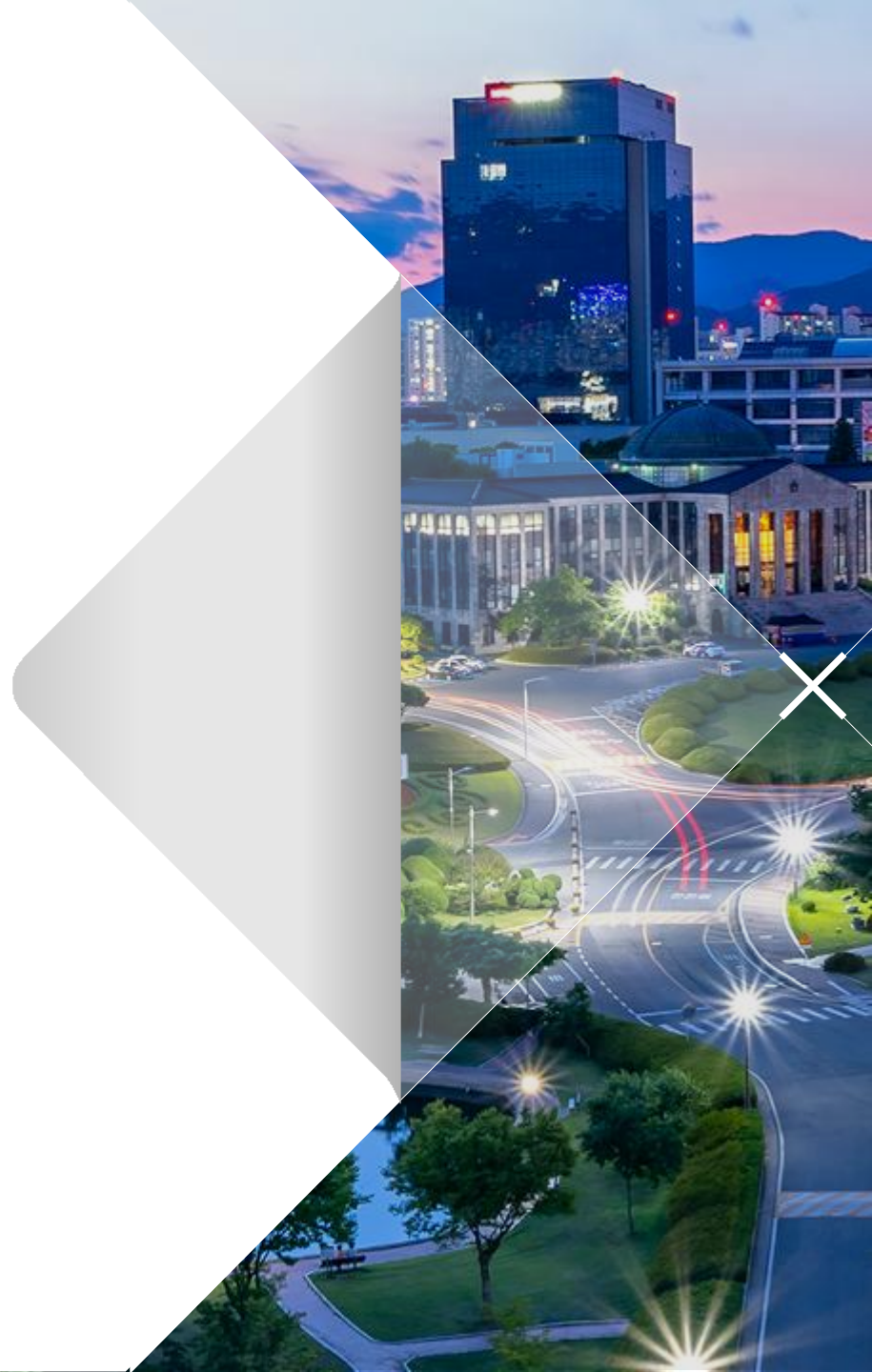
Sunmean Kim

CONTACT

Kyungpook National University
School of Electronics Engineering
Lab. <https://ice.knu.ac.kr>
E-mail sunmean@knu.ac.kr



I. 대회 개요



KNU IDEC CCDC (창의회로설계 챌린지)

➤ <http://ccdc-knuidec.or.kr/>

2. 디지털시스템 설계

주제 : 인공지능 반도체 시스템 (NPU) 설계 경진대회

목표 : 에너지 효율 (TOPS/W)적인 인공지능 반도체 시스템 구조 개발.
오픈소스 EDA 툴 (OpenROAD)과 FinFET 공정 (7nm)을 이용한 디지털 회로 설계 (HDL, Netlist)
및 인공지능 추론 정확도 검증.

- Verilog를 이용한 디지털시스템 설계 (시뮬레이션을 통한 검증)
 - 디지털시스템 설계 분야에는 재료비(10만원) 지급이 되지 않습니다
- ※과제 예시(Project Sample)는 홈페이지 공지사항 참조

시상내역

시상구분	내용	상금
대상 (경북대학교 총장상)	1팀	100만원
금상 (KAIST IDEC 소장상)	2팀	70만원
은상 (KAIST IDEC 소장상)	2팀	50만원
동상 (경북대학교 IDEC 센터장상)	4팀	팀당 30만원
장려상 (Cadence Awards)	10팀	팀당 20만원
산학협력상		-

대회 진행방법

<예선>

- ① 5월 31일 까지: 온라인 접수 (재학증명서, 과제 계획서 첨부)
2인 1조 or 3인 1조
지도교수 1인 반드시 포함
지도교수는 다수 팀 동시 지원 가능
- ② 7월 중: OrCAD PSpice 회로 해석, PCB 설계 동영상 공개
- ③ 8월 31일 까지: 1차 과제보고서(중간보고서) 제출 / 본선 진출팀 선정

<본선>

- ④ 9월 ~ 11월: 선정팀 시작품 제작
시작품 제작 비용 최대 10만원 내 지원금 제공 (부품 구매 영수증 제출 필수)
*지원금 제공 후 영수증 제출, 영수증은 결과 발표일에 제출 혹은 스캔하여 파일로 별도 제출. (영수증 관리에 유의하여 주시기 바랍니다.)

*단, 2025년 디지털시스템 설계 참가자는 재료비(10만원)가 없으니, 착오없으시기 바랍니다.
- ⑤ 10월 31일(토): 과제 결과보고서 PPT 제출
- ⑥ 11월 13일(금): 발표 및 시상

수행 과제

- 에너지 효율(TOPS/W)이 높은 MNIST 분류용 CNN 추론 가속기 설계
- 제공되는 레퍼런스(Baseline) RTL을 출발점으로, 동일 분류 정확도를 유지(또는 향상)하면서 다음을 달성
 - 동작주파수(Fmax) 향상(Timing Slack 개선)
 - 전력(Power) 절감
 - 면적(Area) 축소
- 블록 구성 · 연산 방식 · 아이디어는 자유롭게 추가/개선

평가 및 점수 산정

➤ 평가 방법

- 기능·정확도 : Vivado 2024.1 — RTL 시뮬레이션 + 합성 후(게이트레벨, ASAP7 std_cell) 시뮬레이션
- 성능(면적·타이밍·전력) : OpenROAD 합성 Report
- 결과보고서 내용과 제출물(.xpr)의 일치 여부를 직접 확인

※ Vivado 2024.1 동일 버전 필수

➤ 점수 산정

구분	배점	정성 평가 항목
공통	20%	보고서 10% · 발표 10%
지정주제	50%	완성도 10% · 독창성 10% · 우수성 20% · 가산점 10%
자유주제	30%	완성도 10% · 독창성 10% · 연계성 10%

※ 지정주제 : 제공된 Reference 대비 성능이 얼마나 향상되었는가

자유주제 : 어떤 독창적인 설계 아이디어로 구현하였는가

일정 및 제출물

➤ 일정

1차 과제보고서(중간보고서) 제출 / 본선 진출팀 선정 : ~8월 31일

최종 결과보고서 PPT 및 산출물 제출 : ~10월 31일

홈페이지 과제 제출란 이용

➤ 제출물

- 1차 과제보고서(중간보고서) : PDF 제출

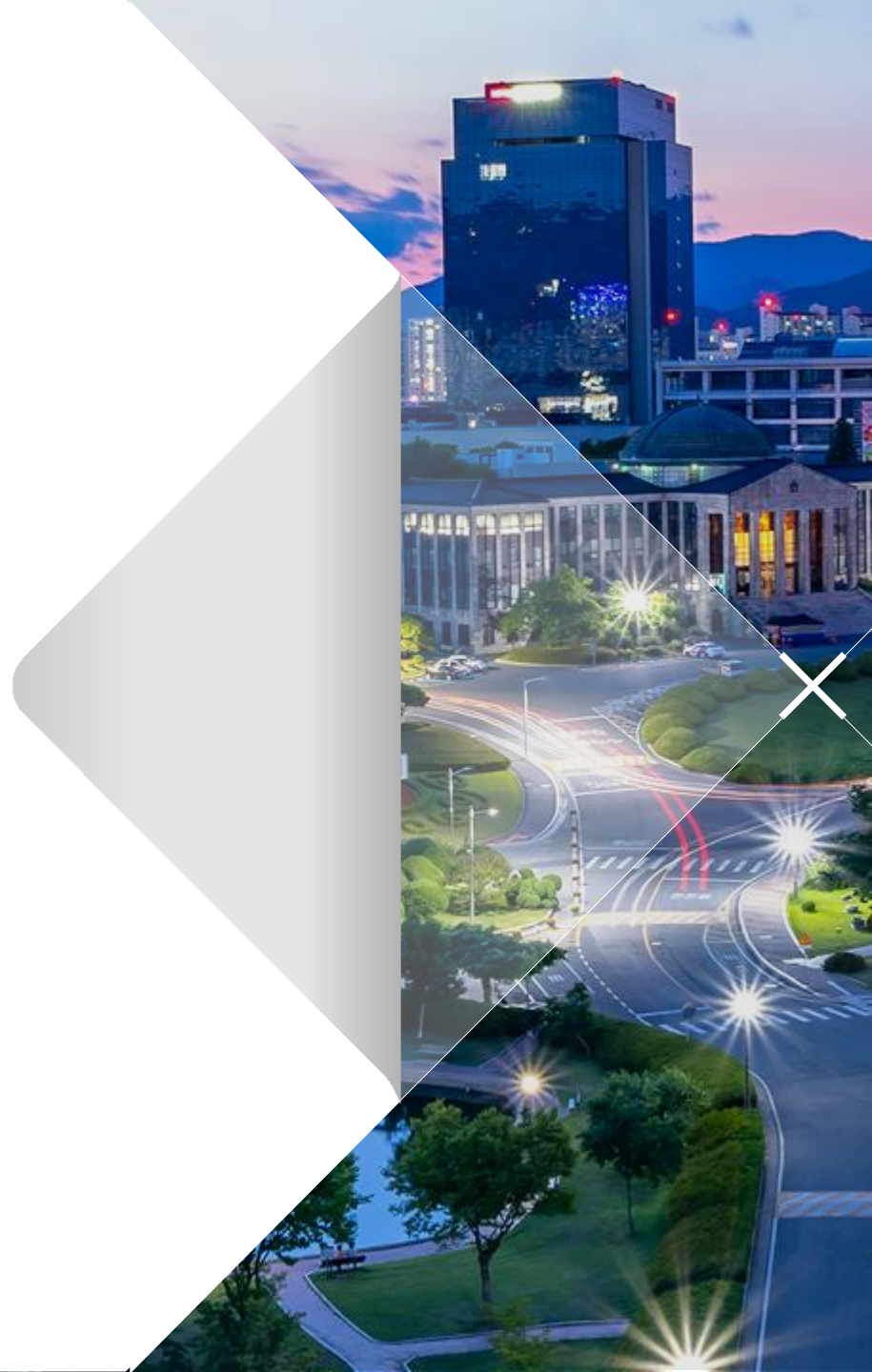
- 최종 결과보고서 및 산출물

1. 결과보고서(.pdf)
2. 프로젝트에 사용한 소스코드 및 테스트벤치 (.v, .txt)
3. RTL 동작 검증 완료한 프로젝트 파일 (.xpr)
4. 합성 netlist 파일 (.v, .sdc), 레포트 파일 (.rpt)
5. 합성 후 동작 검증 완료한 프로젝트 파일 (.xpr)

제공 파일 구성

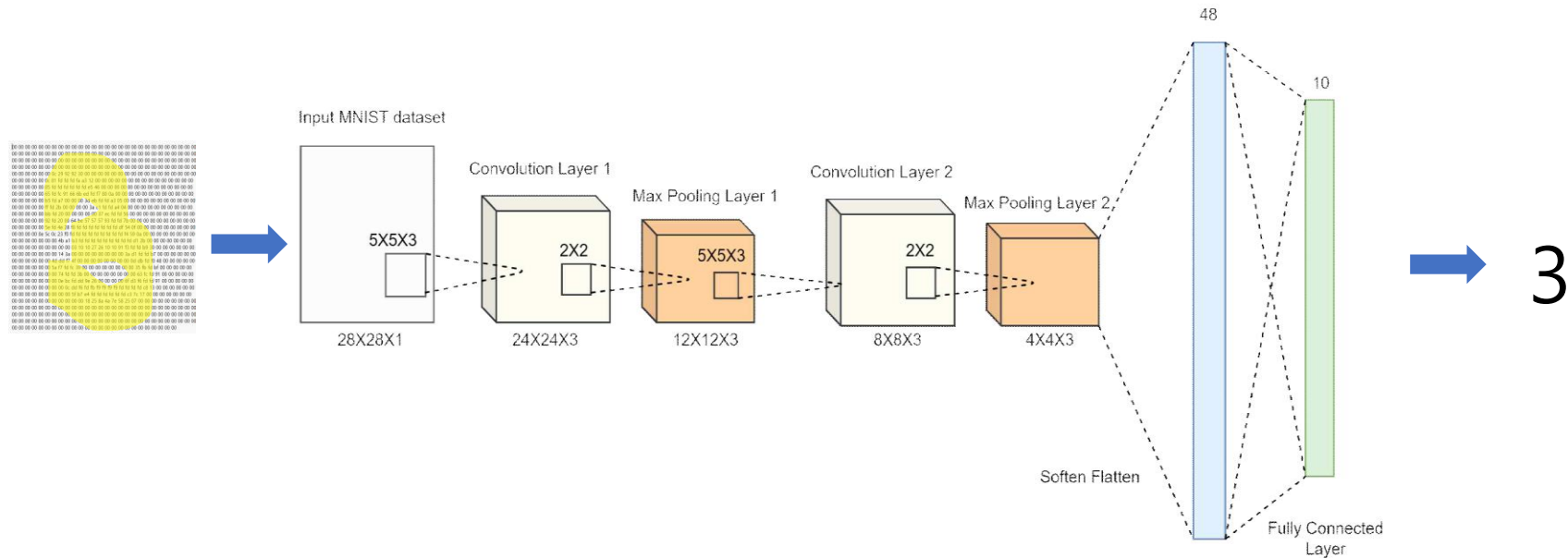
폴더 / 파일	내용
Reference code/cnn_verilog/baseline/	Vivado 프로젝트(baseline.xpr 등) — 더블클릭해 바로 RTL 시뮬레이션 실행
Reference code/cnn_verilog/data/	테스트 입력 이미지(0~9) · conv·FC 가중치/바이어스 · input_1000.txt(정확도 1000장)
Reference code/cnn_verilog/verilog/	RTL 소스(chip·conv1·conv2·fc·maxpool_relu·comparator) + 테스트벤치(top_tb)
OpenRoad project/config/	OpenROAD 합성 설정 — config.mk, constraint.sdc(타이밍 제약), README 등
OpenRoad project/std_cell/	ASAP7 7nm 표준셀(.v) — AO·INVBUF·OA·SEQ·SIMPLE + empty
2026 KNU IDEC CCDC Sample Flow.pptx	설계 → 시뮬레이션 → OpenROAD 합성까지 전체 과제 흐름을 보여주는 예시 가이드

II. Neural Network



Convolution Neural Network

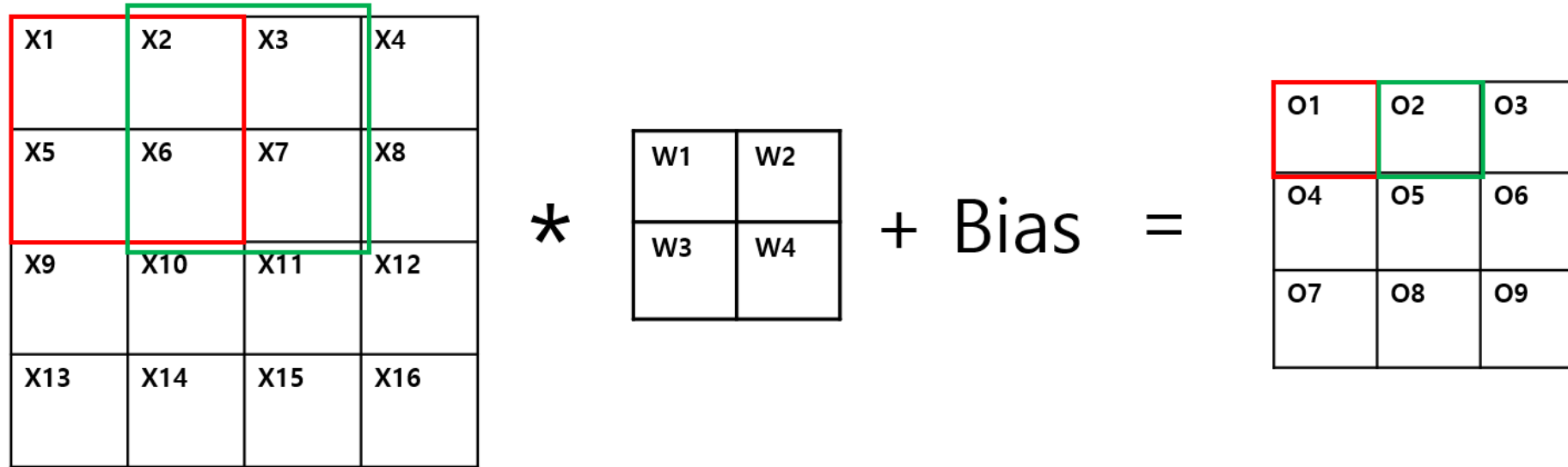
- 주로 이미지 인식·영상 처리 분야에서 사용됨
- 데이터의 공간적 특징(spatial feature)를 추출하는 데 강점이 있음



https://youtu.be/wrguEHxk_EI?si=Q_9MzdQj8k5TbiU_

Convolution

- MAC Operation (Multiply –Accumulation)

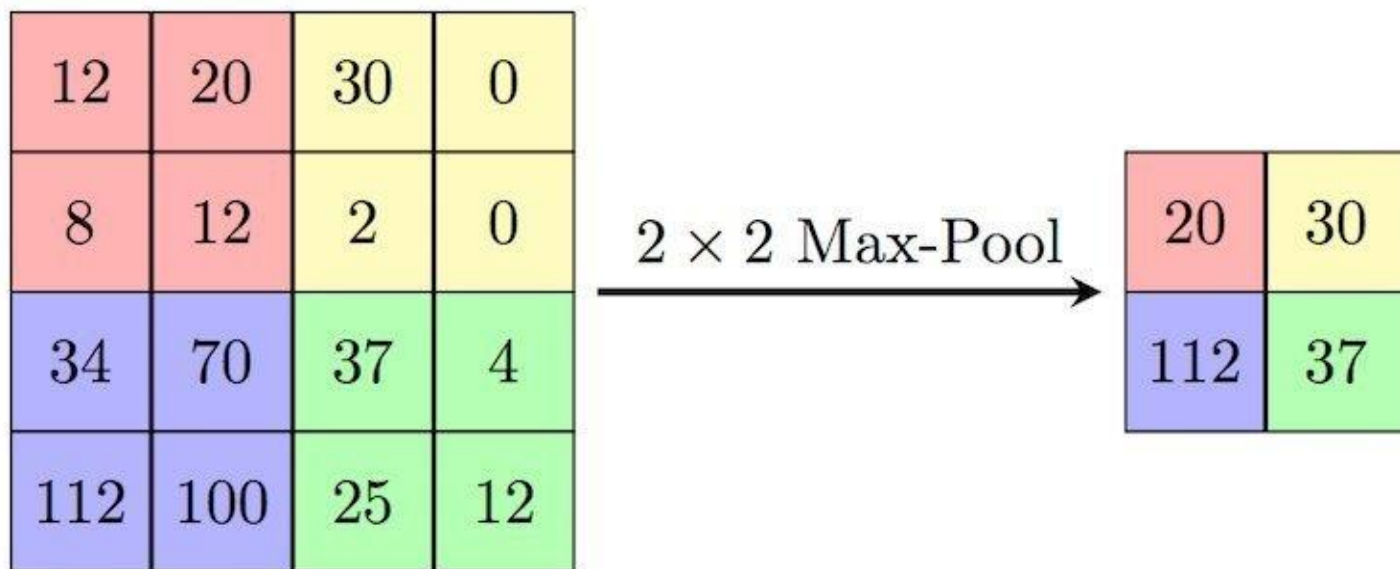


$$O1 = X1 * W1 + X2 * W2 + X5 * W3 + X6 * W4 + \text{Bias}$$

$$O2 = X2 * W1 + X3 * W2 + X6 * W3 + X7 * W4 + \text{Bias}$$

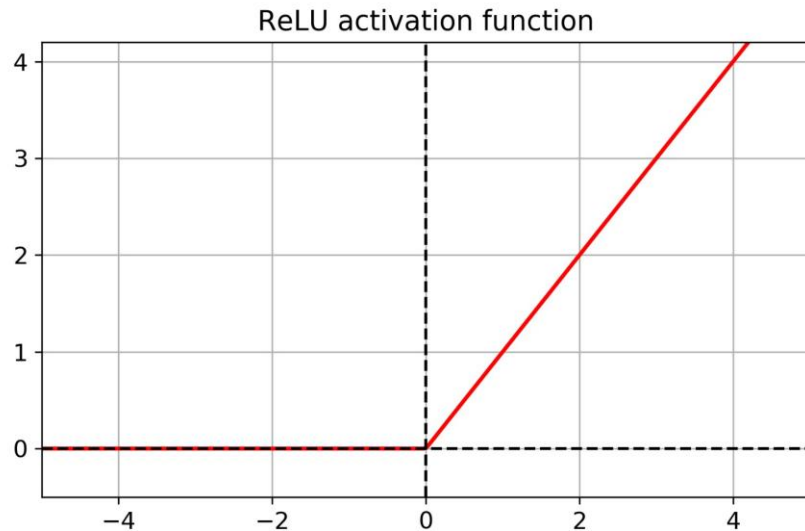
Pooling

- 데이터 크기를 줄여 연산 효율 향상 → 메모리 요구량 감소
- 대표적으로 Max Pooling / Average Pooling이 있음



Activation Function

- 선형 함수를 쓰면 층을 여러 개 쌓아도 1-layer 네트워크와 같아지므로, activation 함수는 반드시 비선형 함수여야 함
- 대표적으로 Sigmoid, Tanh, Relu, Leaky ReLU 등이 있음



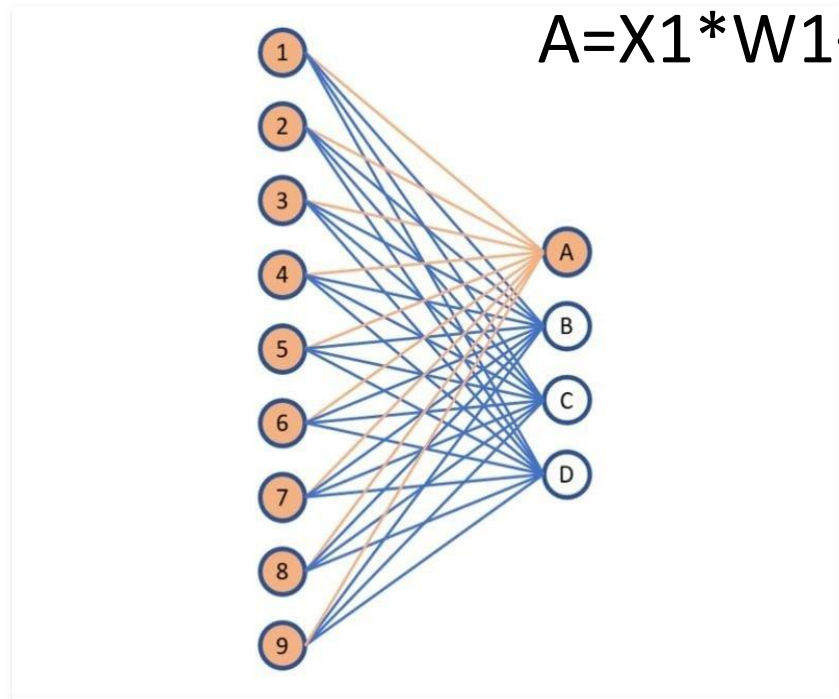
Activation function : $h(x)=c*x$

3 layer network

$$\begin{aligned}\text{Output : } y(x) &= h(h(h(x))) = c*c*c*x \\ &= a*x\end{aligned}$$

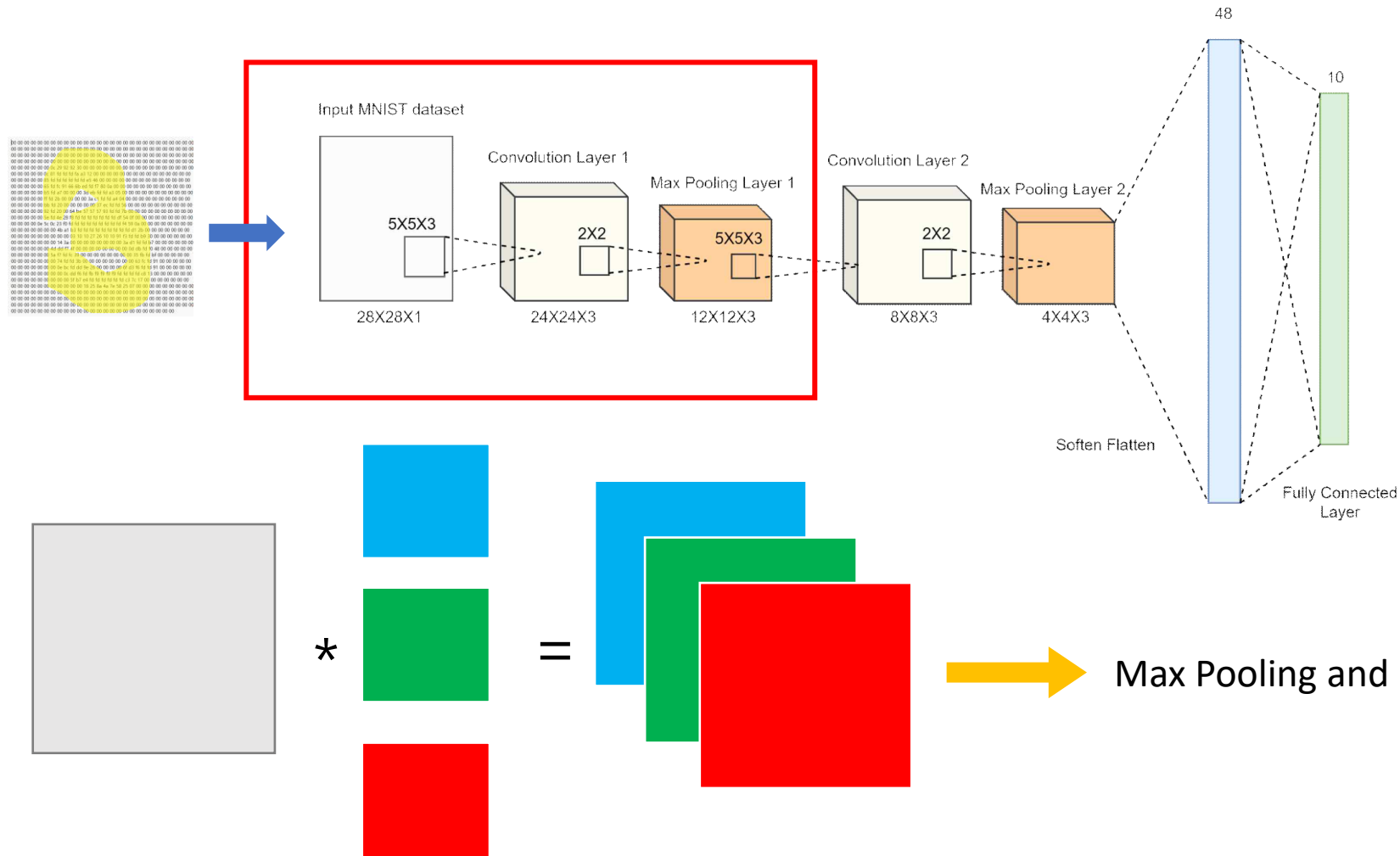
Full Connected Layer

- 이전 Layer의 모든 노드가 다음 Layer의 모든 노드와 연결됨
- 2차원 배열 형태의 이미지를 1차원으로 펼쳐(flatten), 정해진 label로 분류하는 데 사용됨

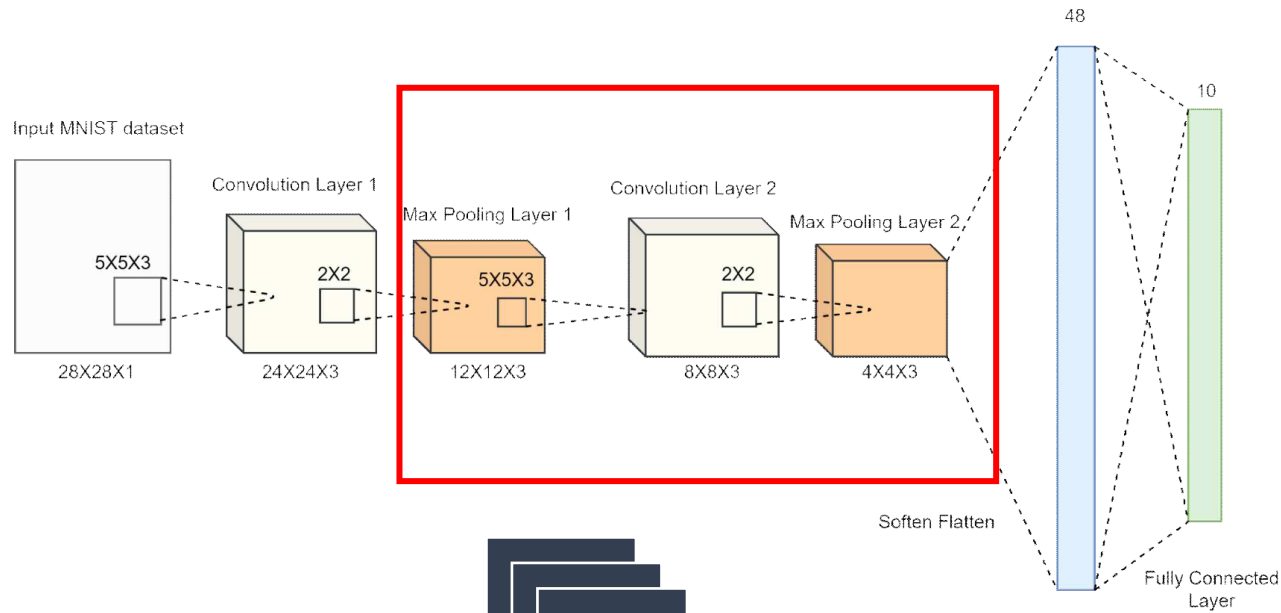


$$A = X1 * W1 + X2 * W2 + X3 * W3 \dots + X9 * W9 + \text{Bias}$$

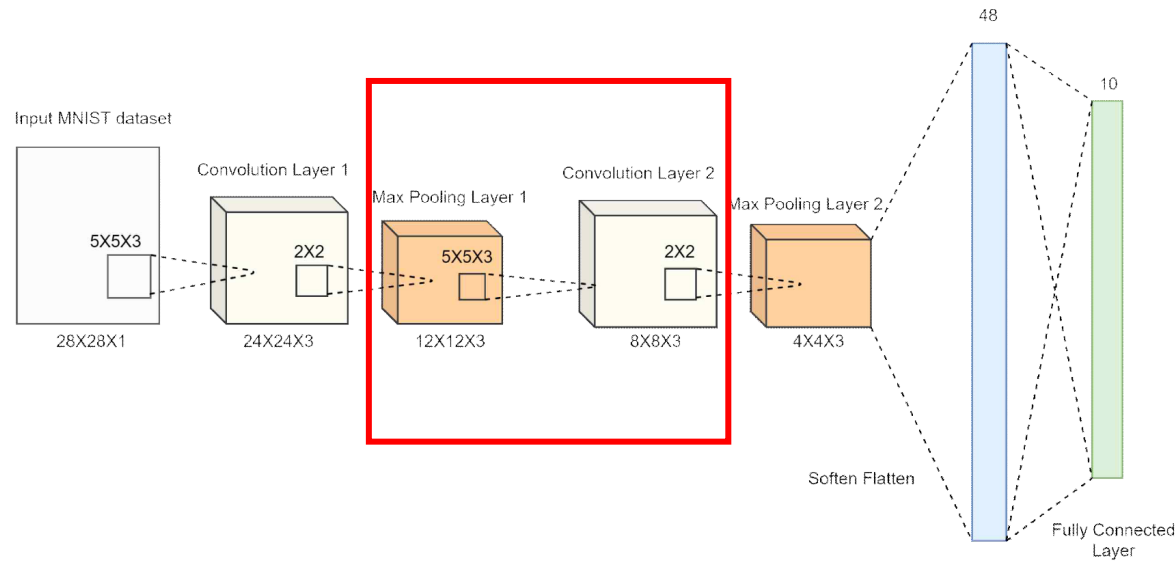
CNN Procedure (1)



CNN Procedure (2)



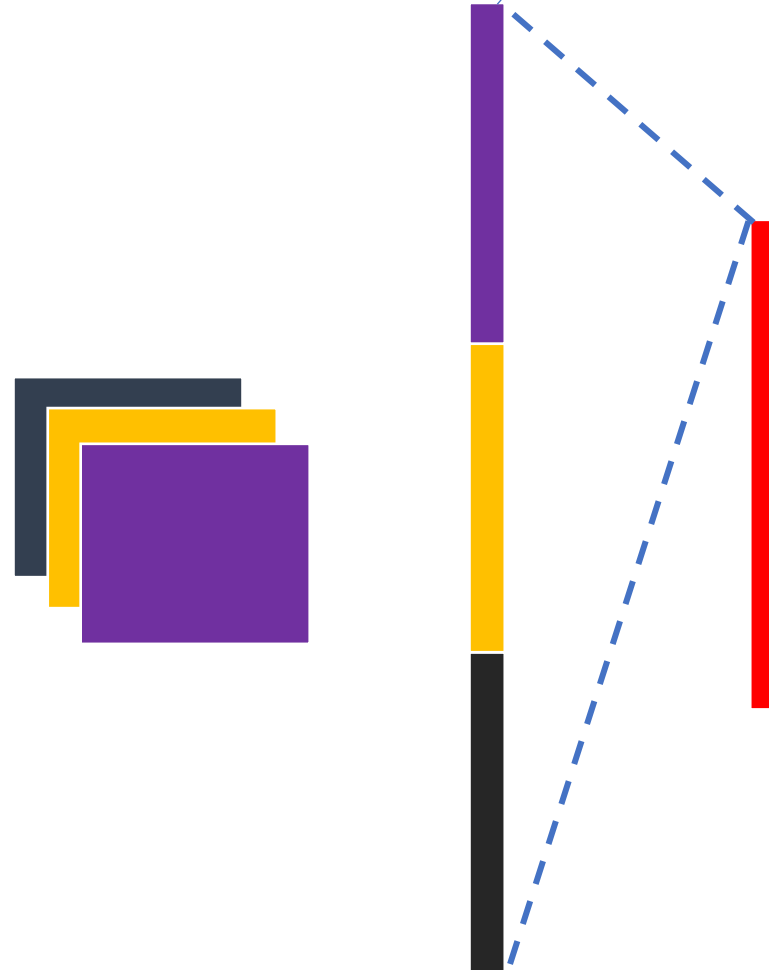
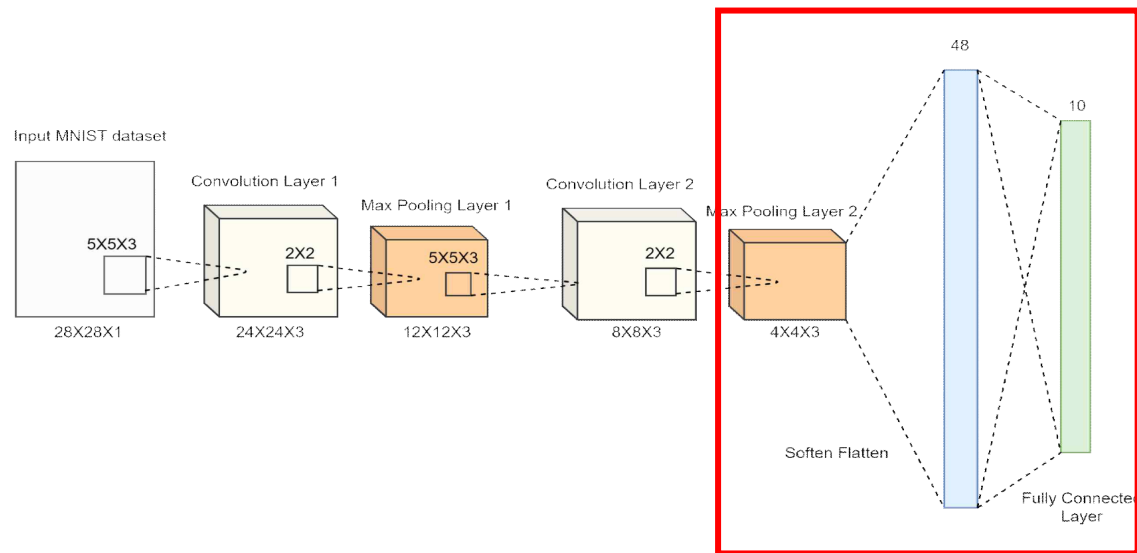
CNN Procedure (3)



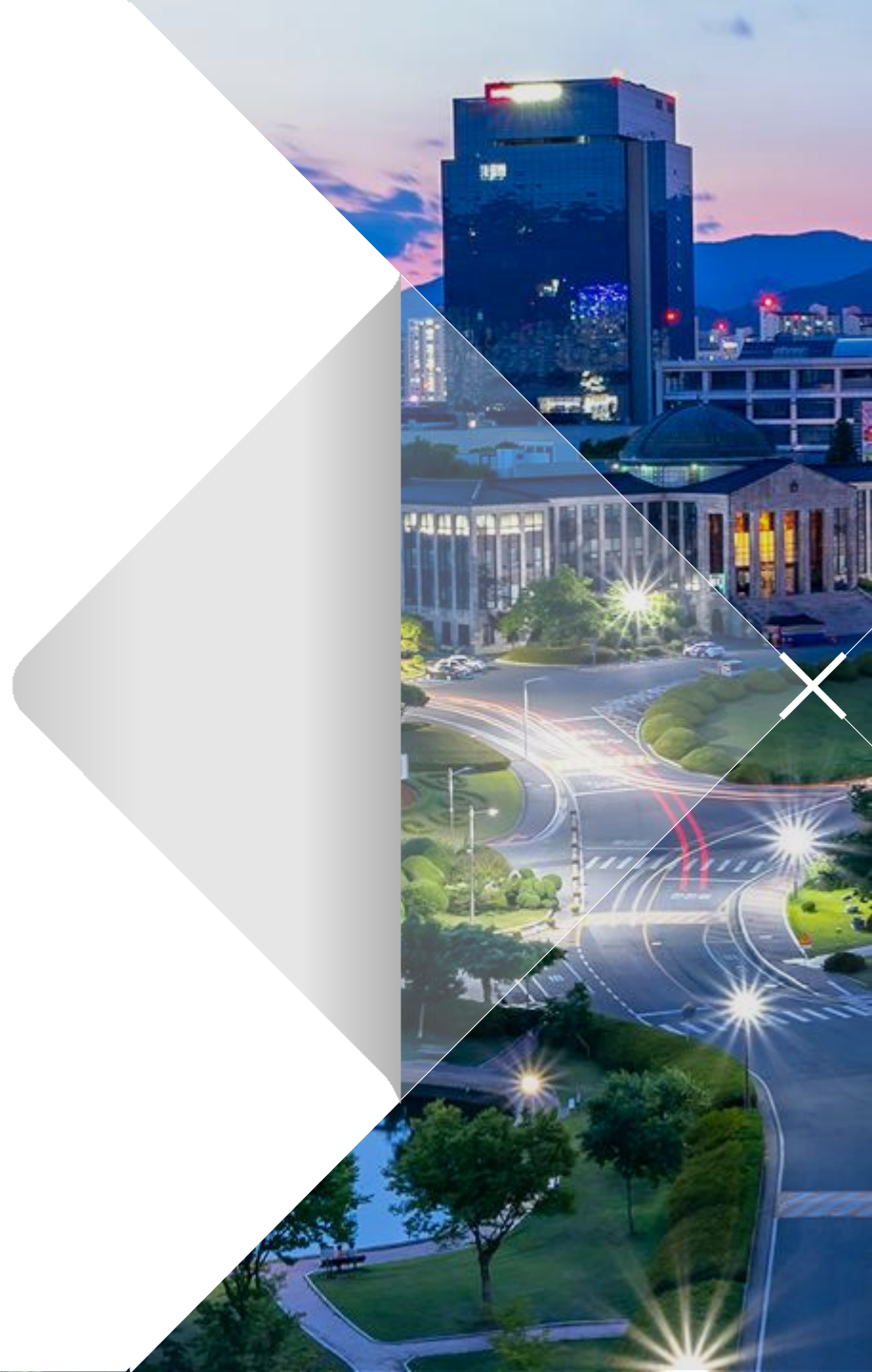
The diagram illustrates the convolution operation. On the left, three colored squares (blue, green, and red) represent the input feature map. These are multiplied by a 3x3 kernel (represented by three dark blue squares with values 3, 2, and 1). The result is a 3x3 output feature map (represented by three colored squares: red, green, and blue). This output is then multiplied by a 3x3 kernel (represented by three dark blue squares with values 1, 2, and 3). The final result is a single dark blue square, representing the output of the convolution operation.

$$\begin{aligned}
 & \begin{bmatrix} \text{Blue} \\ \text{Green} \\ \text{Red} \end{bmatrix} * \begin{bmatrix} 3 \\ 2 \\ 1 \end{bmatrix} = \begin{bmatrix} \text{Red} \\ \text{Green} \\ \text{Blue} \end{bmatrix} \\
 & \begin{bmatrix} \text{Red} \\ \text{Green} \\ \text{Blue} \end{bmatrix} * \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} = \begin{bmatrix} \text{Dark Blue} \end{bmatrix}
 \end{aligned}$$

CNN Procedure (4)



III. CNN Accelerator



Reference Code of CNN Accelerator

➤ <https://github.com/boaaaang/CNN-Implementation-in-Verilog>

The screenshot shows the GitHub repository page for 'boaaaang / CNN-Implementation-in-Verilog'. The repository is public and has 17 forks and 55 stars. The main content area displays the repository's structure, including a 'cnn_verilog' directory, 'docs', 'pyTorch', and a 'README.md' file. A 'Clone' dropdown menu is open, showing options to clone using HTTPS, GitHub CLI, or the web URL, as well as options to open with GitHub Desktop or download a ZIP file. The 'About' section on the right provides a brief description of the repository as a 'Convolutional Neural Network RTL-level Design' and lists related topics like 'neural-network', 'image-processing', 'cnn', 'pytorch', 'mnist', and 'verilog'. The 'Releases' and 'Packages' sections indicate that no releases or packages have been published. The 'Languages' section shows a bar chart indicating the repository is primarily Verilog (70.2%) with some Python (27.8%) and Stata (2.0%).

CNN Implementation in Verilog

Implementation of Convolution Neural Network using Python & Verilog

성균관대학교 소프트웨어학과 & 전자전기공학부 강보영 졸업작품

Environment Setting

Python Installment

<https://www.python.org/>

About

Convolutional Neural Network RTL-level Design

neural-network image-processing cnn
pytorch mnist verilog

Readme
Activity
55 stars
2 watching
17 forks
Report repository

Releases

No releases published

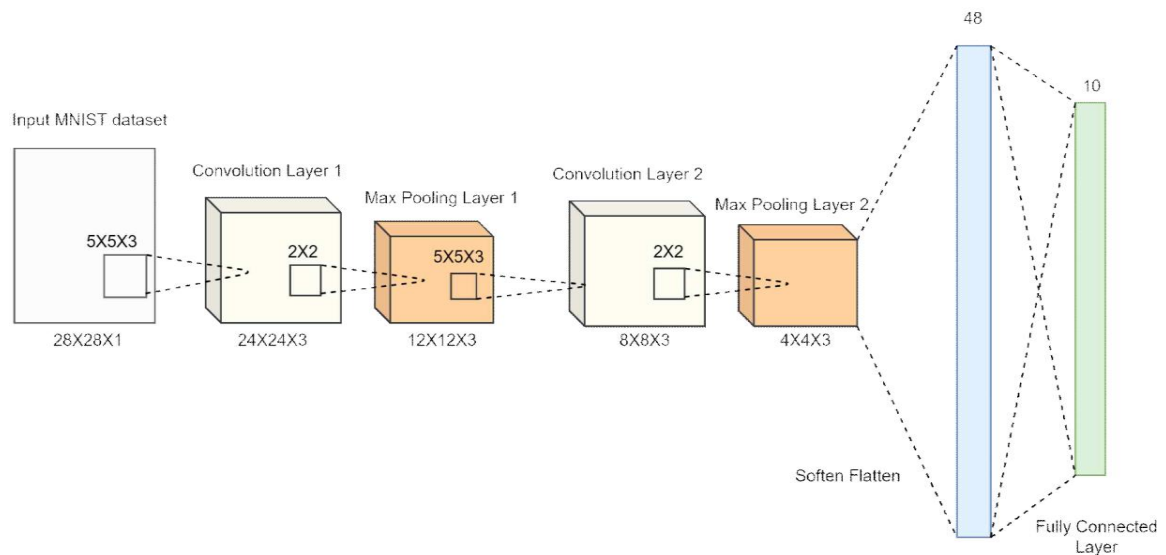
Packages

No packages published

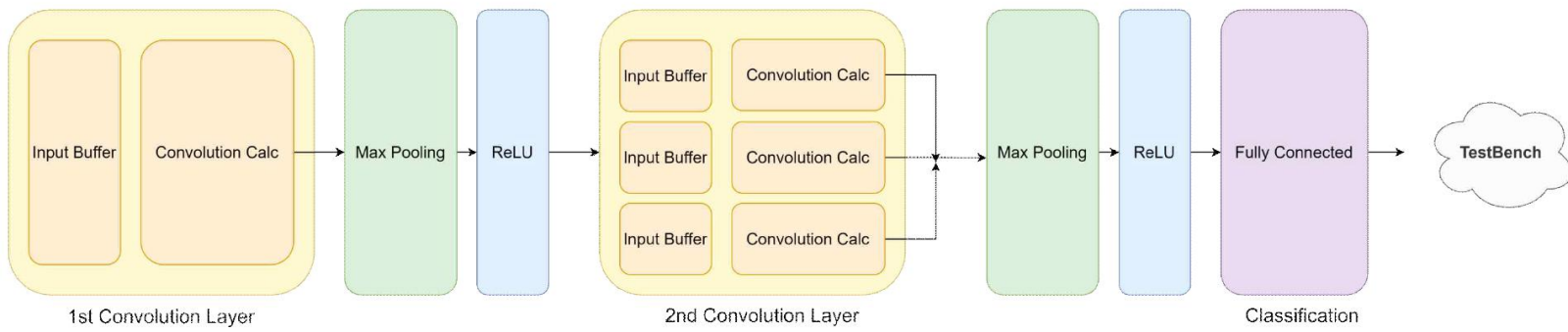
Languages

Verilog 70.2% Python 27.8%
Stata 2.0%

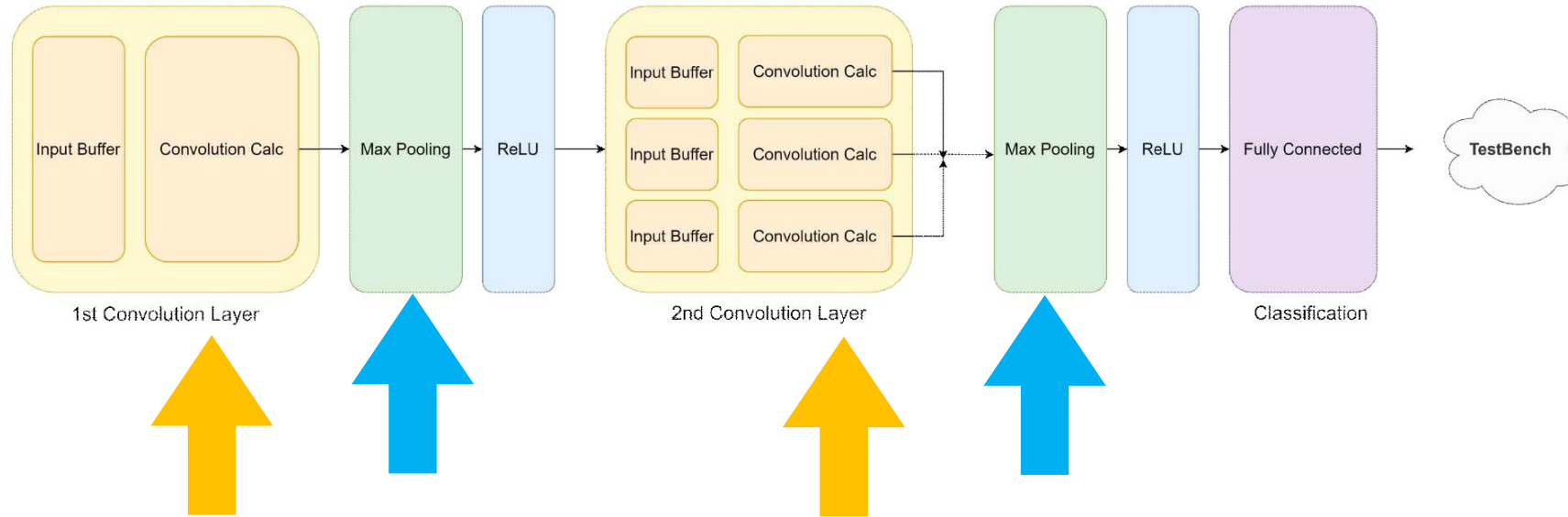
Reference Code: Architecture (1)



```
top_tb (top_tb.v) (6)
> conv1_layer : conv1_layer (conv1_layer.v) (2)
  maxpool_relu_1 : maxpool_relu (maxpool_relu.v)
> conv2_layer : conv2_layer (conv2_layer.v) (6)
  maxpool_relu_2 : maxpool_relu (maxpool_relu.v)
  fully_connected : fully_connected (fully_connected.v)
  comparator : comparator (comparator.v)
```

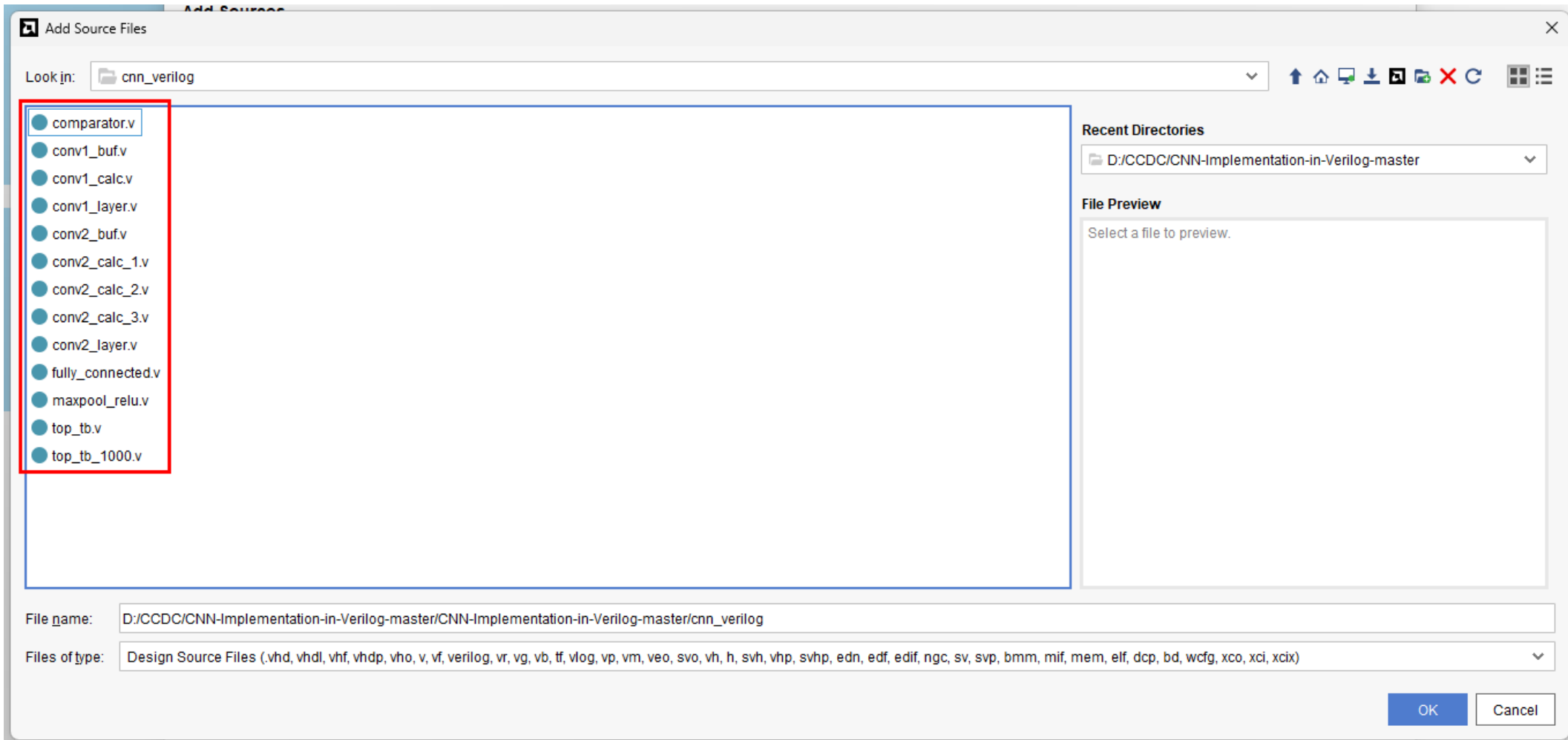


Reference Code: Architecture (2)



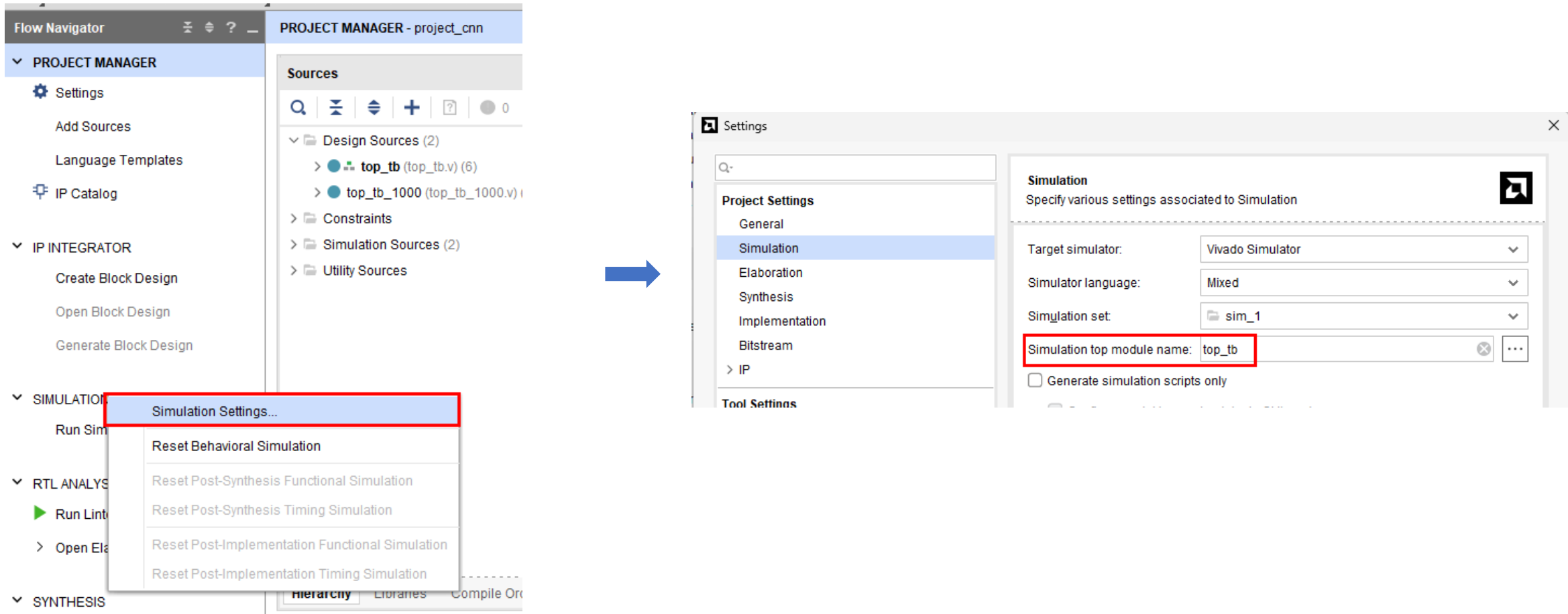
- 같은 convolution · Pooling 연산을 별도 모듈에서 반복 수행 → 면적 낭비
- 데이터 크기에 종속적인(유연하지 못한) CNN 모델

Reference Code: RTL Simulation (1)

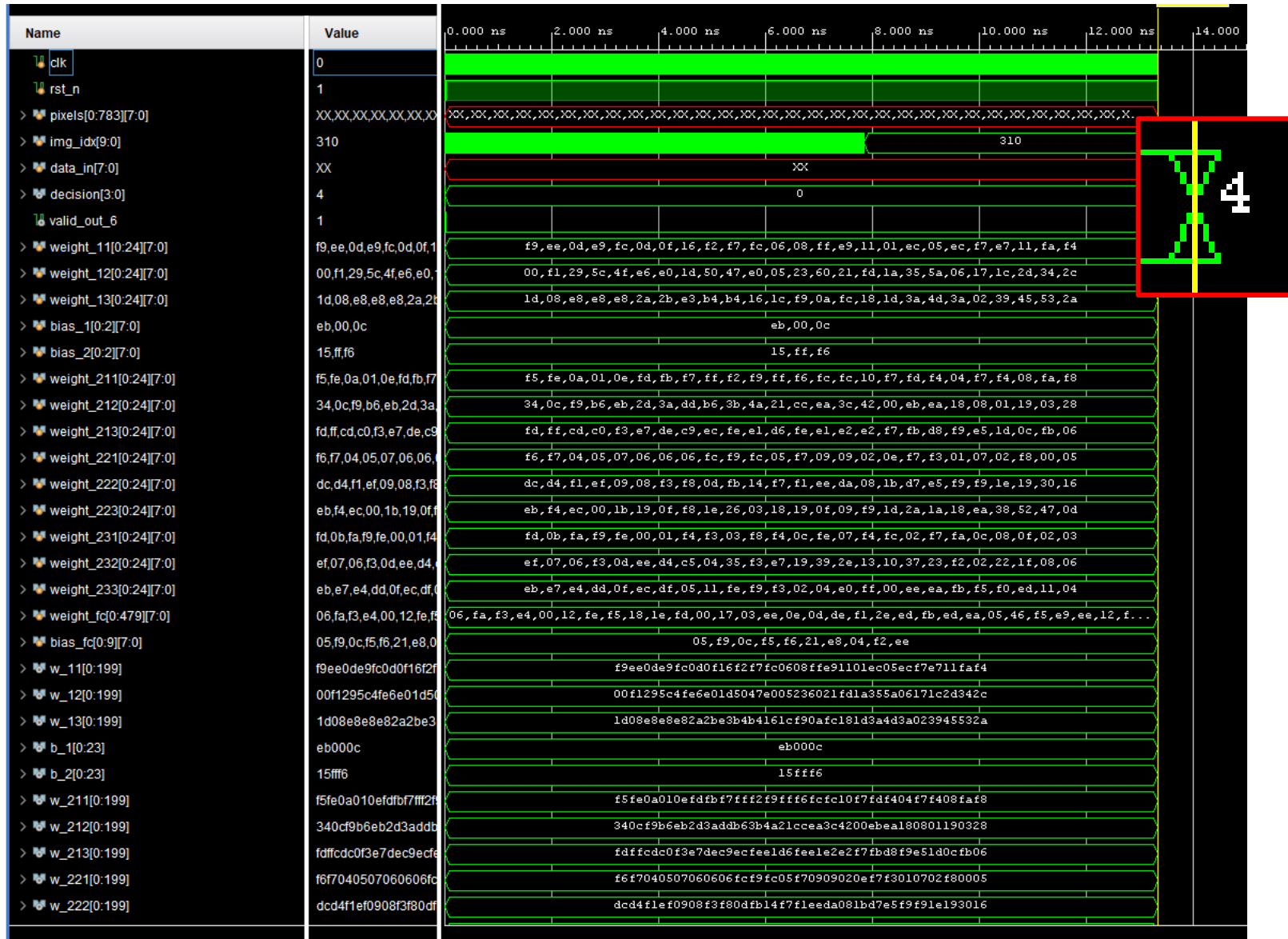


Reference Code: RTL Simulation (2)

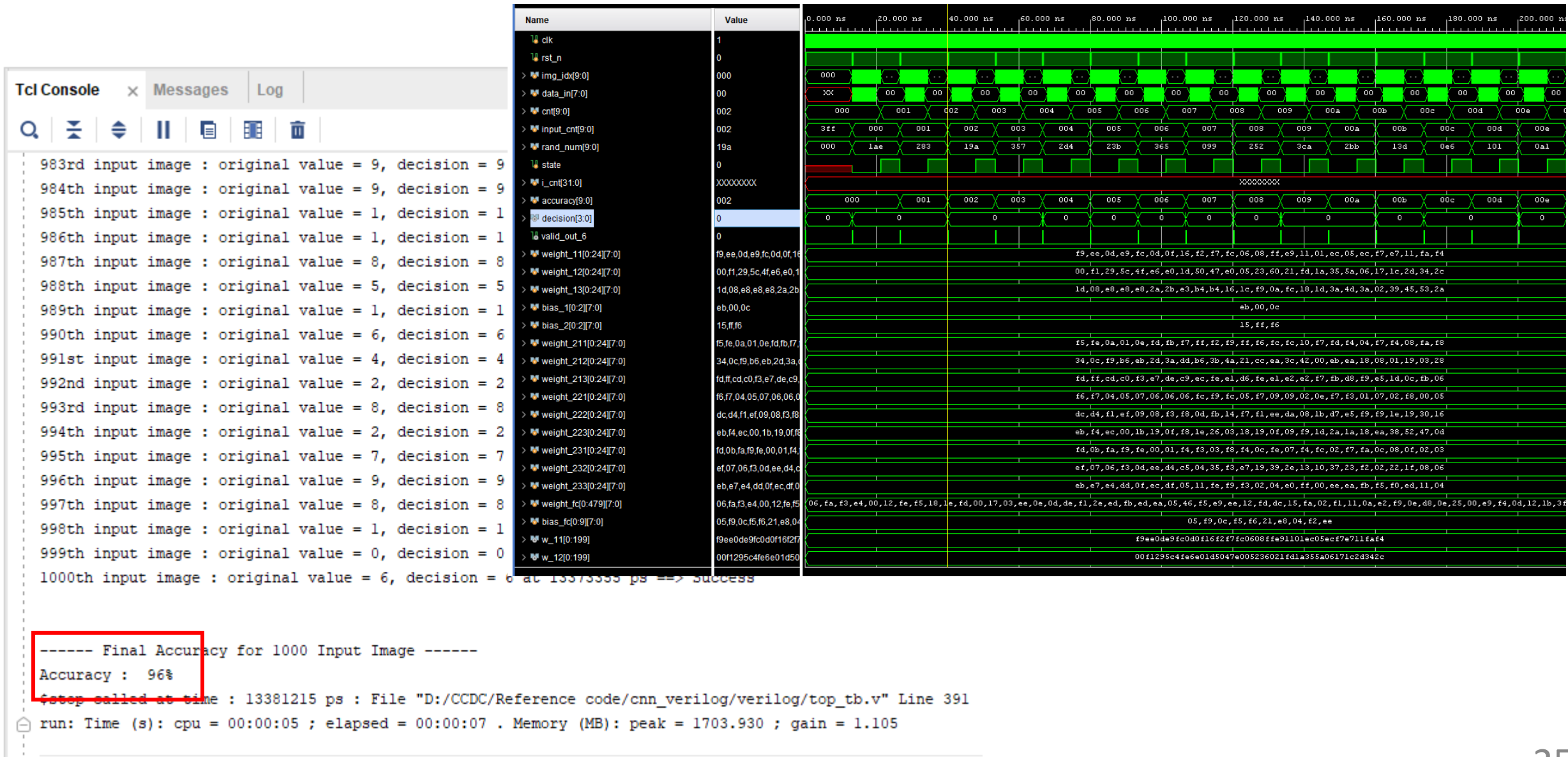
- **Top_tb** : 하나의 입력 이미지에 대한 분류 시뮬레이션
- **Top_tb_1000** : 임의의 1000개 이미지에 대한 시뮬레이션 (Accuracy 출력)



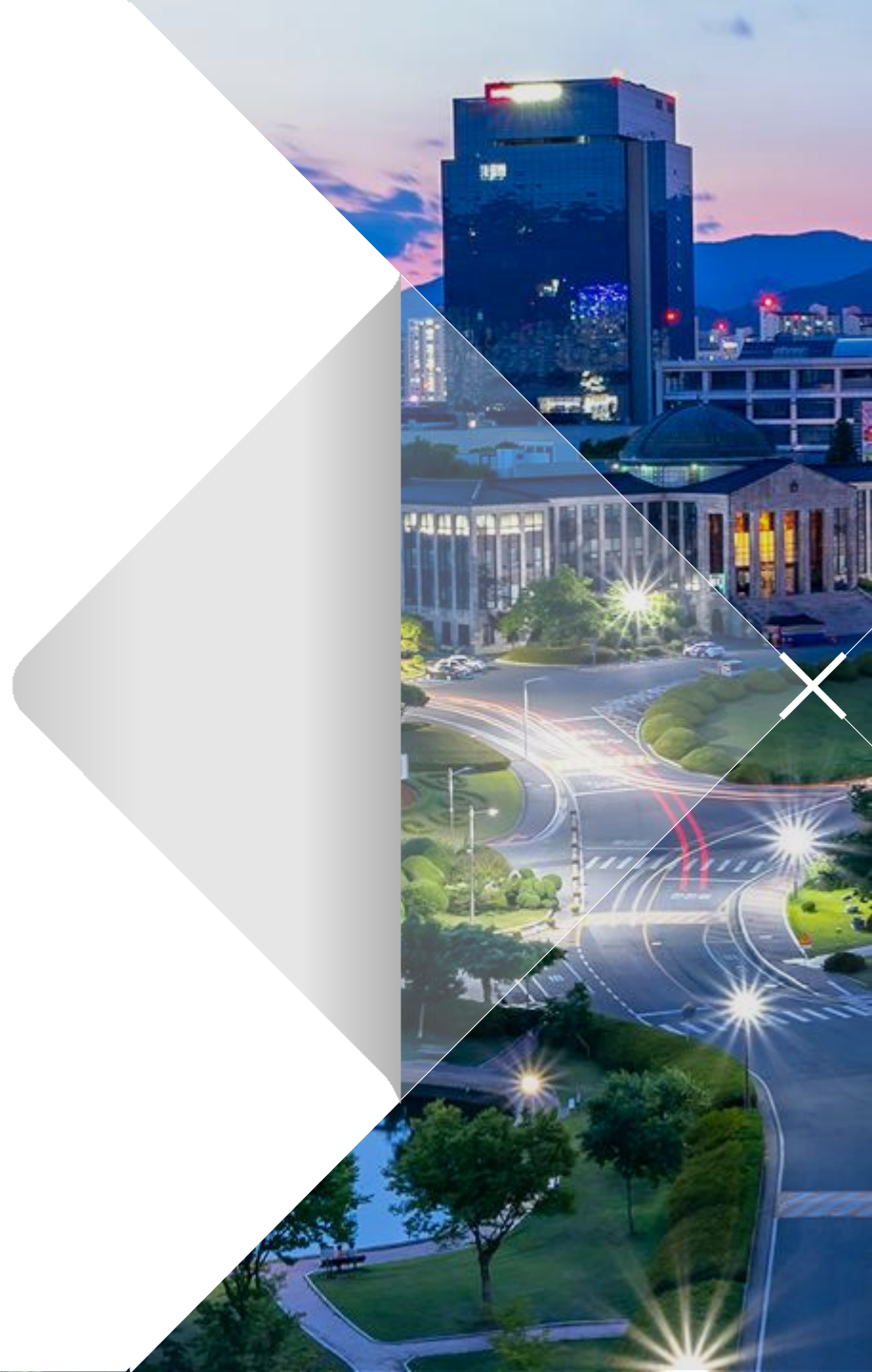
Reference Code: RTL Simulation Result (TOP_tb)



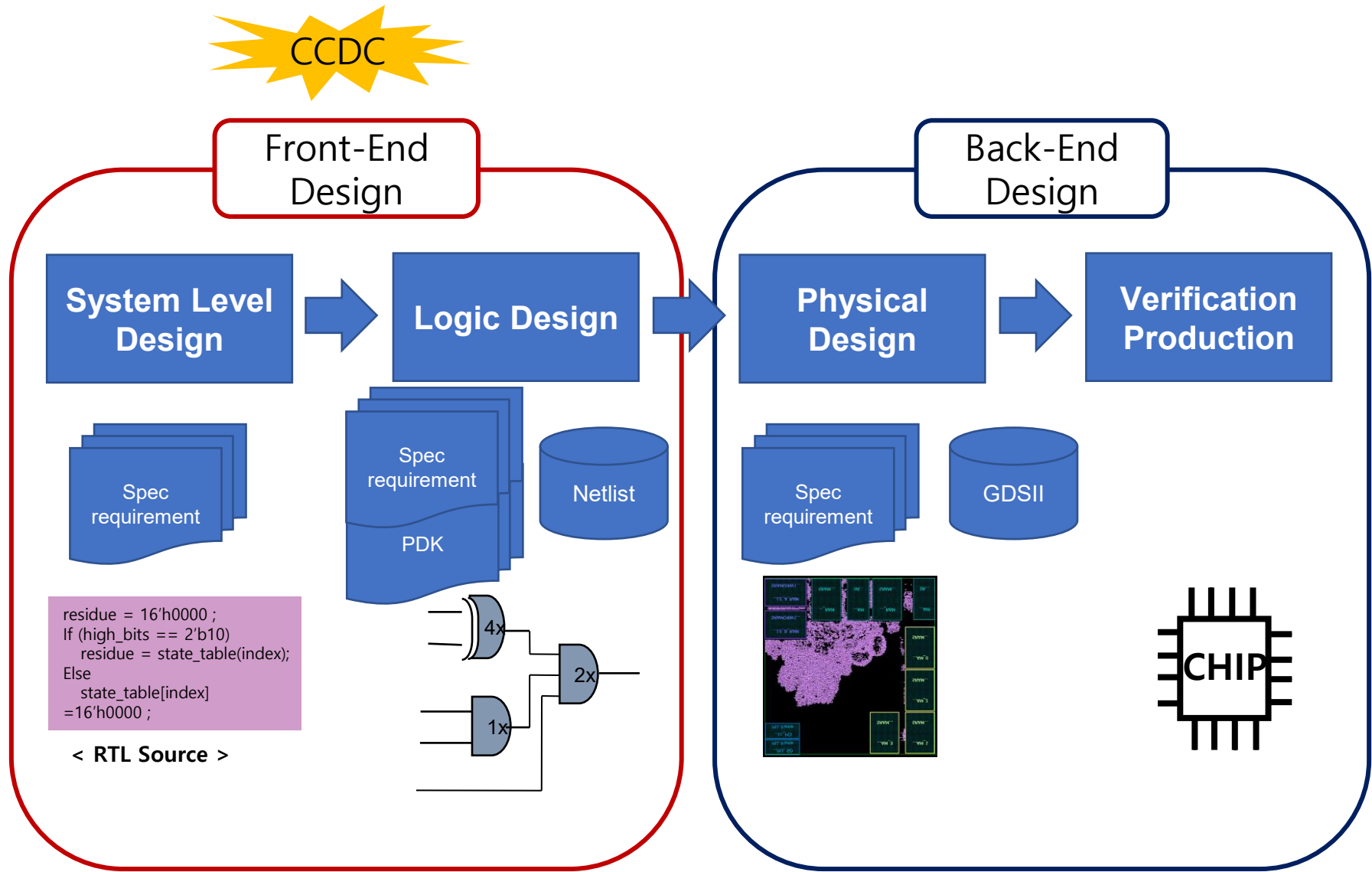
Reference Code: RTL Simulation Result (TOP_tb_1000)



IV. ASIC Design Flow



Digital ASIC Design Flow



Logic Design (Synthesis)

RTL (.v)

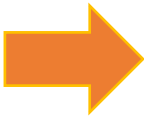
```
module conv1_calc #(parameter WIDTH = 28, HEIGHT = 28, DATA_BITS = 8)(
    input valid_out_buf,
    input [DATA_BITS - 1:0] data_out_0, data_out_1, data_out_2, data_out_3, data_out_4,
    data_out_5, data_out_6, data_out_7, data_out_8, data_out_9,
    data_out_10, data_out_11, data_out_12, data_out_13, data_out_14,
    data_out_15, data_out_16, data_out_17, data_out_18, data_out_19,
    data_out_20, data_out_21, data_out_22, data_out_23, data_out_24,
    output signed [11:0] conv_out_1, conv_out_2, conv_out_3,
    output reg valid_out_calc
);

localparam FILTER_SIZE = 5;
localparam CHANNEL_LEN = 3;

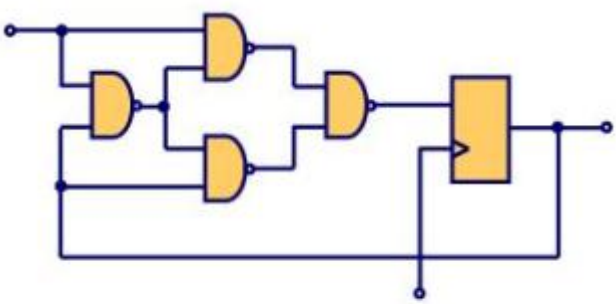
reg signed [DATA_BITS - 1:0] weight_1 [0:FILTER_SIZE * FILTER_SIZE - 1];
reg signed [DATA_BITS - 1:0] weight_2 [0:FILTER_SIZE * FILTER_SIZE - 1];
reg signed [DATA_BITS - 1:0] weight_3 [0:FILTER_SIZE * FILTER_SIZE - 1];
reg signed [DATA_BITS - 1:0] bias [0:CHANNEL_LEN - 1];

wire signed [19:0] calc_out_1, calc_out_2, calc_out_3;
wire signed [DATA_BITS:0] exp_data [0:FILTER_SIZE * FILTER_SIZE - 1];
wire signed [11:0] exp_bias [0:CHANNEL_LEN - 1];

initial begin
    $readmemh("conv1_weight_1.txt", weight_1);
    $readmemh("conv1_weight_2.txt", weight_2);
    $readmemh("conv1_weight_3.txt", weight_3);
    $readmemh("conv1_bias.txt", bias);
end
```



Netlist (.v)



Area, Delay, Power report

Post synthesis report_power					
Group	Internal Power	Switching Power	Leakage Power	Total Power (Watts)	
Sequential	6.07e-05	2.77e-06	2.48e-09	6.35e-05	29.4%
Combinational	5.95e-05	9.26e-05	2.48e-08	1.52e-04	70.6%
Clock	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.0%
Macro	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.0%
Pad	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.0%
Total	1.20e-04	9.54e-05	2.73e-08	2.16e-04	100.0%
	55.7%	44.2%	0.0%		

Logic Synthesis

➤ 환경 : Ubuntu 22.04

- MobaXterm install : rtlog.tistory.com/entry/MobaXterm-설치-방법-WSL-연동
- Ubuntu 22.04 LTS download : <https://apps.microsoft.com/detail/9pn20msr04dw?launch=true&mode=full&hl=ko-kr&gl=kr&ocid=bingwebsearch>
- MobaXterm Ubuntu 사용 : <https://fossa.tistory.com/7> (환경변수 설정은 필요 없음)

➤ Tools : Vivado(2024.1)

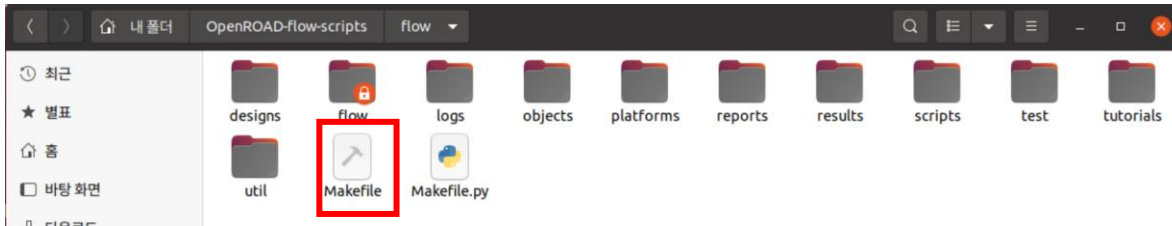
- Download : <https://www.amd.com/en/support/downloads/adaptive-socs-and-fpgas/development-tools/2024-1.html>

➤ Tools : OpenROAD

- Download : <https://openroad-flow-scripts.readthedocs.io/en/latest/>
- Build Manual : [Build from sources using Docker — OpenROAD Flow documentation \(openroad-flow-scripts.readthedocs.io\)](https://openroad-flow-scripts.readthedocs.io/en/latest/build-manual.html)

How to use OpenROAD Project

➤ https://m.blog.naver.com/gc_na/223351182438



OpenROAD-flow-scripts/flow/Makefile

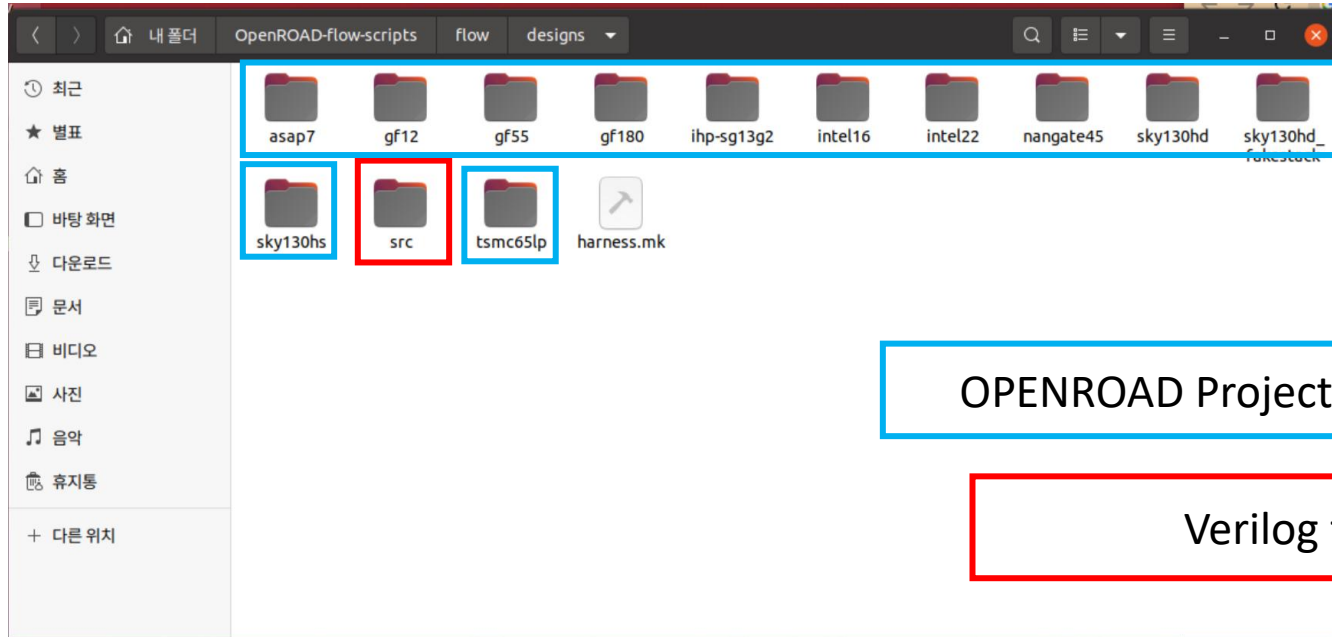
```
506 $(RESULTS_DIR)/1_1_yosys.v: $(SDC_FILE_CLOCK_PERIOD)
507     $(UNSET_AND_MAKE) do-yosys
508
509 $(RESULTS_DIR)/1_synth.sdc: $(SDC_FILE)
510     mkdir -p $(REPORTS_DIR)
511     cp $(SDC_FILE) $(RESULTS_DIR)/1_synth.sdc
512
513 .PHONY: do-synth
514 do-synth:
515     mkdir -p $(RESULTS_DIR) $(LOG_DIR) $(REPORTS_DIR)
516     cp $(RESULTS_DIR)/1_1_yosys.v $(RESULTS_DIR)/1_synth.v
517
518 $(RESULTS_DIR)/1_synth.v: $(RESULTS_DIR)/1_1_yosys.v
519     $(UNSET_AND_MAKE) do-synth
```

```
503 $(RESULTS_DIR)/1_1_yosys.v: $(SDC_FILE_CLOCK_PERIOD)
504     $(UNSET_AND_MAKE) do-yosys
505
506 $(RESULTS_DIR)/1_synth.sdc: $(SDC_FILE)
507     mkdir -p $(REPORTS_DIR)
508     cp $(SDC_FILE) $(RESULTS_DIR)/1_synth.sdc
509
510 .PHONY: do-synth
511 do-synth:
512     mkdir -p $(RESULTS_DIR) $(LOG_DIR) $(REPORTS_DIR)
513     cp $(RESULTS_DIR)/1_1_yosys.v $(RESULTS_DIR)/1_synth.v
514     $(UNSET_AND_MAKE) synth-report
```

추가

For report about Netlist after synthesis

How to use OpenROAD Project



OPENROAD Project에서 제공하는 공정

Verilog file 저장 위치

How to use OpenROAD Project

The image displays three screenshots of the OpenROAD project file structure, illustrating the locations for Verilog files, project processes, and execution files.

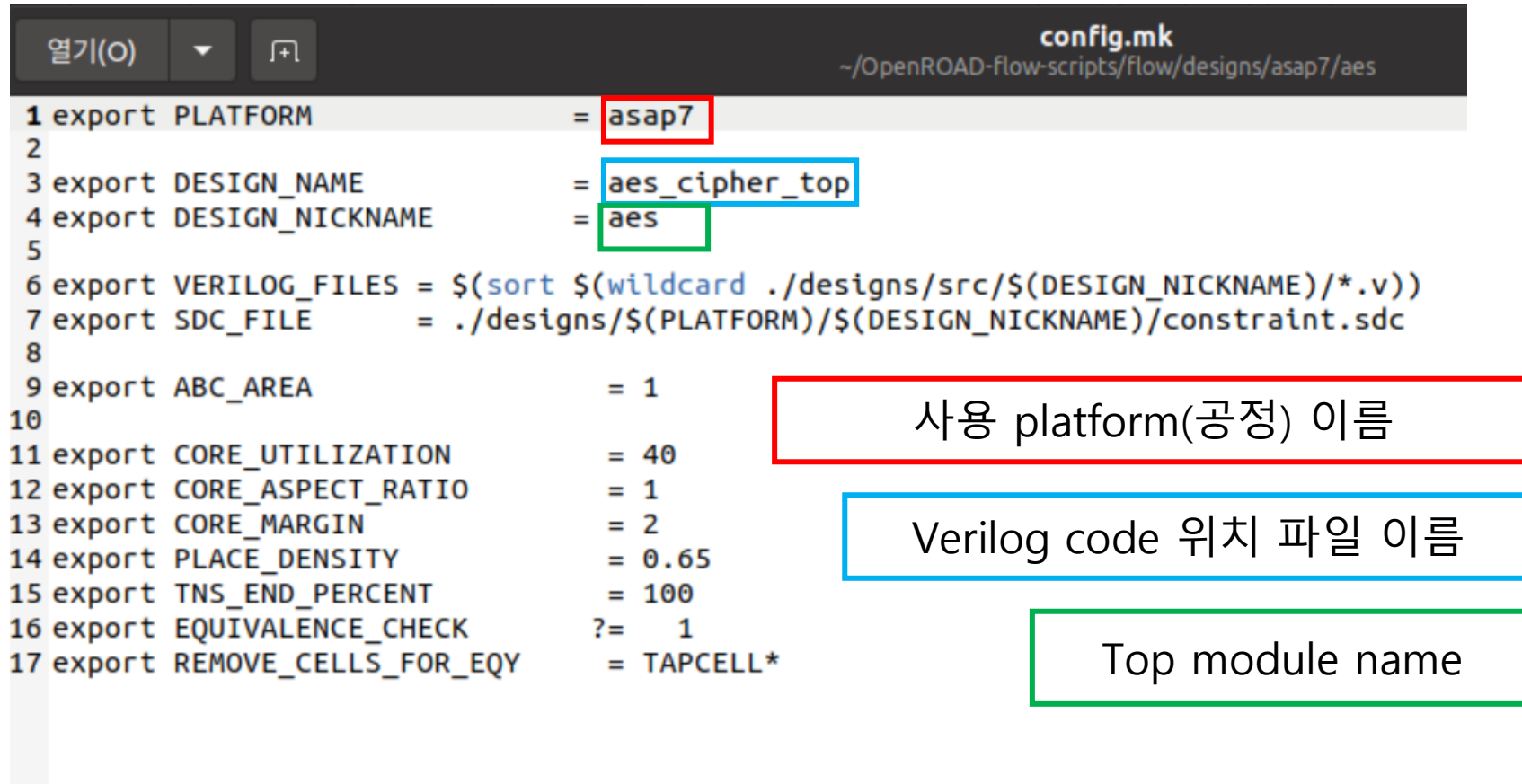
Verilog file 저장 위치 (Verilog file storage location): This annotation points to the `src` folder in the `OpenROAD-flow-scripts` directory.

OPENROAD Project에서 제공하는 공정 (Processes provided by the OpenROAD Project): This annotation points to the `src` folder in the `OpenROAD-flow-scripts` directory.

OpenROAD 실행 file 위치 (OpenROAD execution file location): This annotation points to the `config.mk` file in the `OpenROAD-flow-scripts` directory.

실행 파일 (Execution file): This annotation points to the `config.mk` file in the `OpenROAD-flow-scripts` directory.

How to use OpenROAD Project

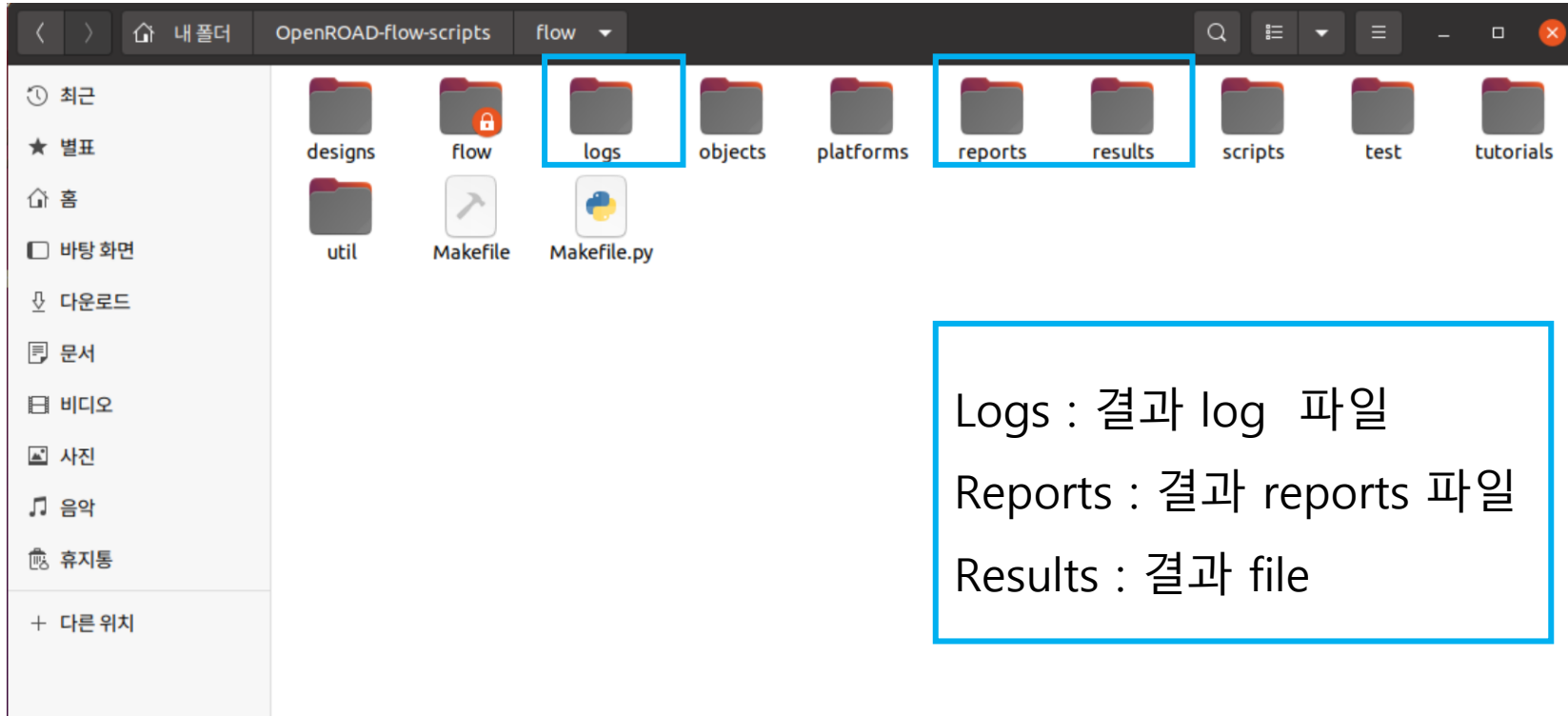


The image shows a screenshot of a text editor window displaying a file named `config.mk` located at `~/OpenROAD-flow-scripts/flow/designs/asap7/aes`. The file contains 17 lines of configuration for an OpenROAD project. Three specific values are highlighted with colored boxes and annotated with Korean text:

- `asap7` (line 1) is highlighted with a red box. A red-bordered callout box points to it with the text "사용 platform(공정) 이름" (Platform name).
- `aes_cipher_top` (line 3) is highlighted with a blue box. A blue-bordered callout box points to it with the text "Verilog code 위치 파일 이름" (Verilog code location file name).
- `aes` (line 4) is highlighted with a green box. A green-bordered callout box points to it with the text "Top module name" (Top module name).

```
1 export PLATFORM = asap7
2
3 export DESIGN_NAME = aes_cipher_top
4 export DESIGN_NICKNAME = aes
5
6 export VERILOG_FILES = $(sort $(wildcard ./designs/src/$(DESIGN_NICKNAME)/*.v))
7 export SDC_FILE = ./designs/$(PLATFORM)/$(DESIGN_NICKNAME)/constraint.sdc
8
9 export ABC_AREA = 1
10
11 export CORE_UTILIZATION = 40
12 export CORE_ASPECT_RATIO = 1
13 export CORE_MARGIN = 2
14 export PLACE_DENSITY = 0.65
15 export TNS_END_PERCENT = 100
16 export EQUIVALENCE_CHECK ?= 1
17 export REMOVE_CELLS_FOR_EQY = TAPCELL*
```

How to use OpenROAD Project



How to use OpenROAD Project

```
/* Generated by Yosys 0.40 (git sha1 UNKNOWN, g++ 11.4.0-1ubuntu)
(* src = "../designs/src/CNNCORE/chip.v:1:1-123:10" *)
(* top = 1 *)
(* hdlname = "chip" *)
module chip(clk, rst_n, data_in, decision, valid_out_6, w_11,
);
  wire _000000;
  wire _000001;
  wire _000002;
  (* src = "../designs/src/CNNCORE/chip.v:1623:2-1655:5" *)
  wire _000003;
  (* src = "../designs/src/CNNCORE/chip.v:1623:2-1655:5" *)
  wire _000004;
  (* src = "../designs/src/CNNCORE/chip.v:1623:2-1655:5" *)
  wire _000005;
  (* src = "../designs/src/CNNCORE/chip.v:1623:2-1655:5" *)
  wire _000006;
  (* src = "../designs/src/CNNCORE/chip.v:1623:2-1655:5" *)
  wire _000007;
  (* src = "../designs/src/CNNCORE/chip.v:1623:2-1655:5" *)
  wire _000008;
  (* src = "../designs/src/CNNCORE/chip.v:1623:2-1655:5" *)
  wire _000009;
  (* src = "../designs/src/CNNCORE/chip.v:1623:2-1655:5" *)
  wire _000010;
  (* src = "../designs/src/CNNCORE/chip.v:1623:2-1655:5" *)
  wire _000011;
  (* src = "../designs/src/CNNCORE/chip.v:1623:2-1655:5" *)
  wire _000012;
  (* src = "../designs/src/CNNCORE/chip.v:1623:2-1655:5" *)
  wire _000013;
  (* src = "../designs/src/CNNCORE/chip.v:1623:2-1655:5" *)
  wire _000014;
  (* src = "../designs/src/CNNCORE/chip.v:1623:2-1655:5" *)
  wire _000015;
  (* src = "../designs/src/CNNCORE/chip.v:1623:2-1655:5" *)
  wire _000016;
  (* src = "../designs/src/CNNCORE/chip.v:1623:2-1655:5" *)
```

Netlist

Post synthesis report_power

Group	Internal Power	Switching Power	Leakage Power	Total Power (Watts)	
Sequential	3.96e-02	3.69e-04	8.64e-07	4.00e-02	3.6%
Combinational	4.73e-01	5.95e-01	1.73e-05	1.07e+00	96.4%
Clock	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.0%
Macro	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.0%
Pad	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.0%
Total	5.12e-01	5.96e-01	1.81e-05	1.11e+00	100.0%
	46.2%	53.8%	0.0%		

report

V. Issue Report

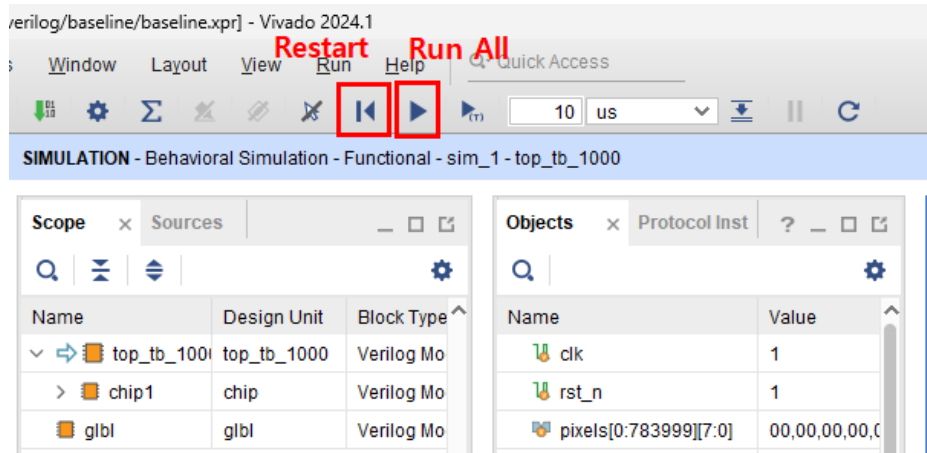


Simulation 시 주의 사항

```
// Read image text file
initial begin
    $readmemh("/home/daegyeom/txt/3_0.txt", pixels);
    clk <= 1'b0;
    rst_n <= 1'b1;
end
```

- \$readmemh 의 파일 경로를
각자 폴더에 맞는 절대 경로로 설정하기

(Ex) "/conv1_weight_1.txt" → "/home/user/CNN-Implementation-in-Verilog/pyTorch/mnist_cnn/conv1_weight_1.txt"

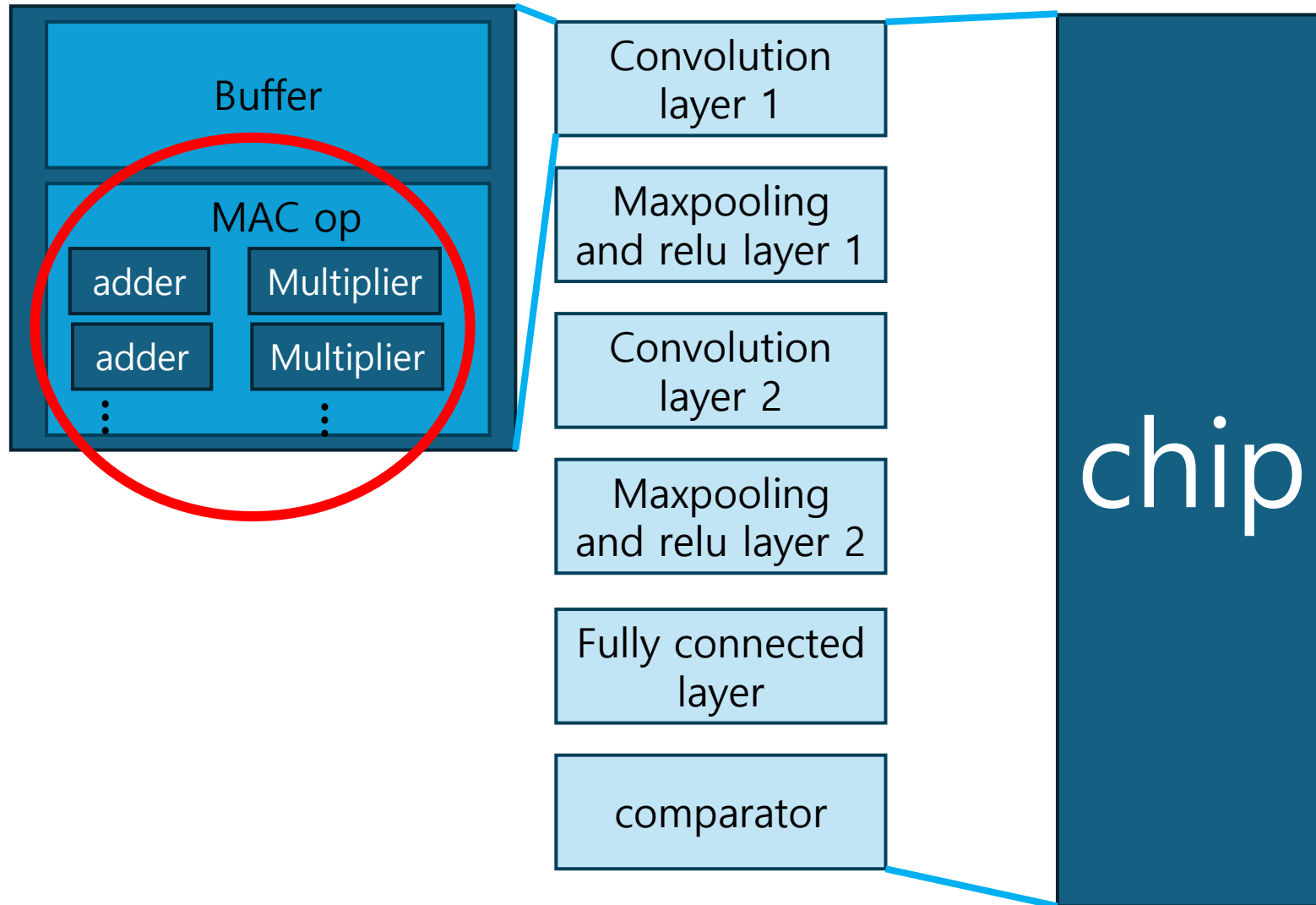


- Top_tb_1000 돌릴 때 1000th 까지 진행 안되면, Restart 후
"Run All" 로 진행하기

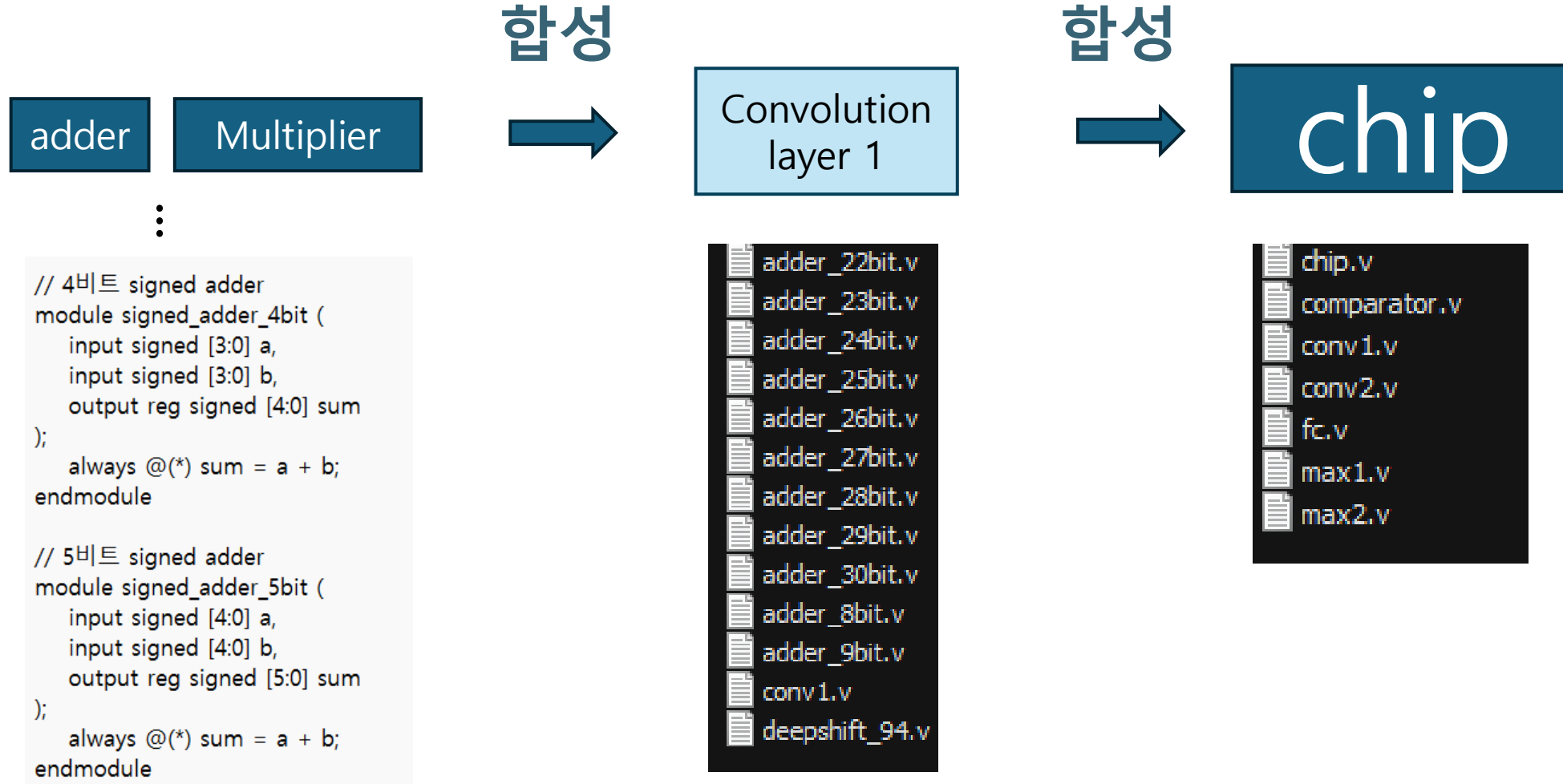
OpenRoad Issues

1. 하위 모듈 인식 못함 → 하위 모듈 합성 후, netlist instance해서 재 합성
2. 모듈 선언에서 parameter 선언하면 제대로 인식 못함 → parameter 별로 따로 합성 후 구분하여 사용
3. If 문 인식에 문제 → 코드 수정

OpenRoad Issues

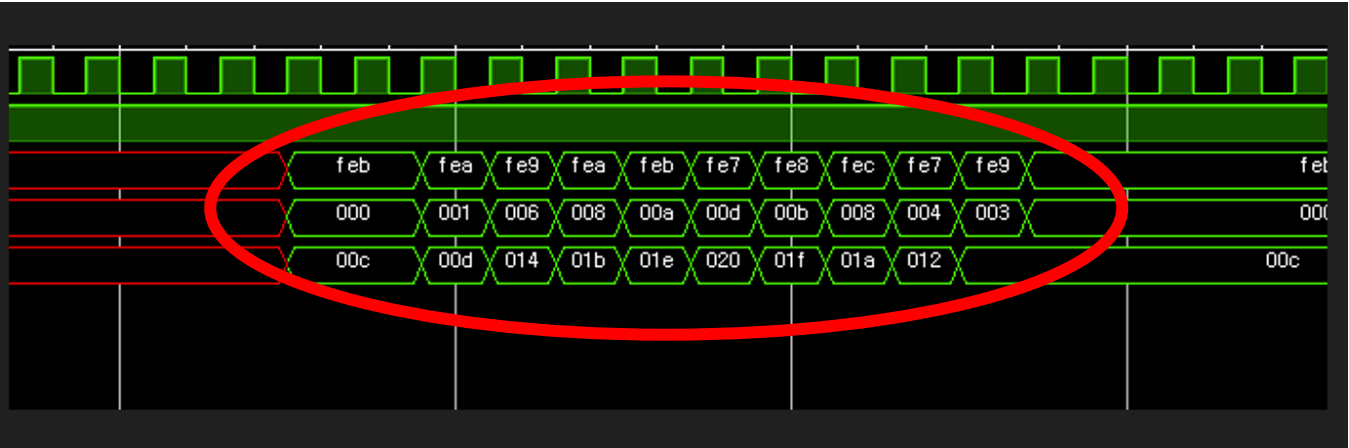


OpenRoad Issues

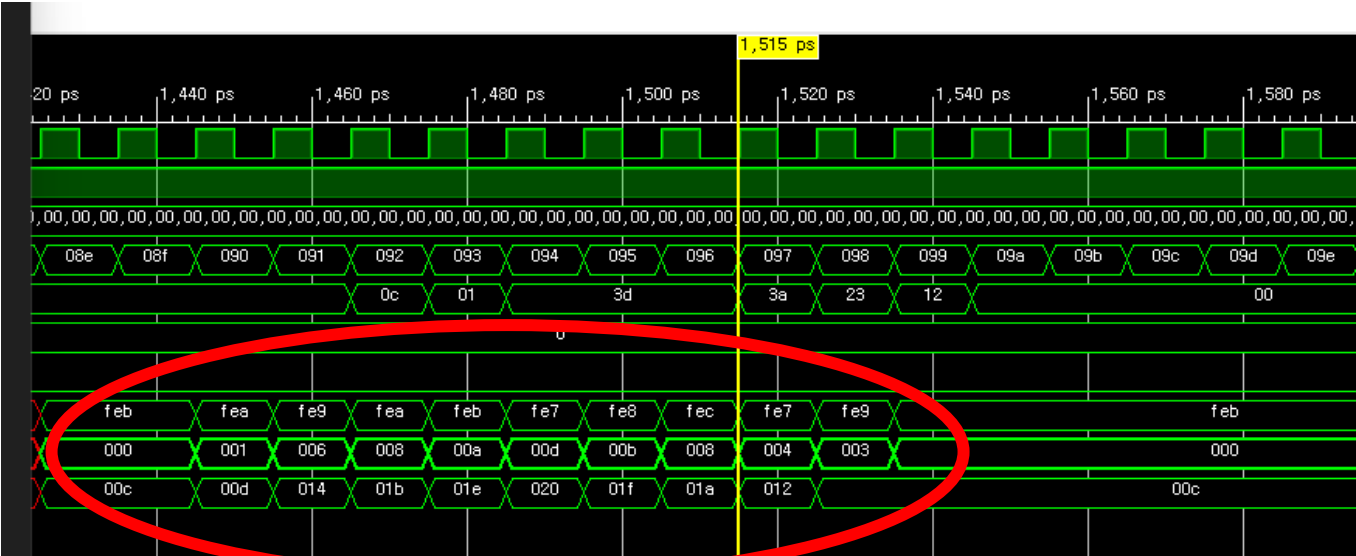


Synth file

OpenRoad Issues



Before
Synthesis

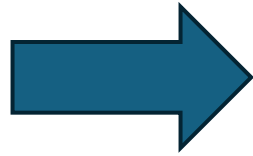


after
Synthesis

OpenRoad Issues

이것만 합성됨

```
If (a)begin
    data_0
End
If (b)begin
    data_1
End
If (c)begin
    data_2
end
```



```
If (a)begin
    data_0
End
Else If (b)begin
    data_1
End
Else If (c)begin
    data_2
end
```

OR

```
data_0 = a ? ~~
data_1 = b ? ~~
data_2 = c ? ~~
```

Thank you

